Supplementary material

fastPLS: accelerated SIMPLS for high-dimensional biomedical data with compiled CPU and accelerator backends

Dupe Ojo[†], Alessia Vignoli[†], Stefano Cacciatore[*], Leonardo Tenori[*]

†These authors contributed equally and share first authorship.

*Co-corresponding authors: Stefano Cacciatore, stefano.cacciatore@icgeb.org; Leonardo Tenori, tenori@cerm.unifi.it.

This supplement distinguishes the current audited fastPLS 0.99.10 interface from the 0.99.6 source archive used for the quantitative benchmarks. Table S15 maps each analysis to its result archive and records an exact package commit only when run metadata captured it. A later package version, result date, or manuscript commit is never treated as evidence of a historical computational SHA.

Route-level evidence is consolidated here rather than repeated in the main text. Table S1 defines stage residency; Tables S7-S7b separate deterministic estimator validation, same-code execution ablation, and the direct PLS-SVD/SIMPLS shape comparison; Table S8 and Figure S1 give rSVD controls, qualification, and quarantined workflow speed; Table S9 is the authoritative float32 capability matrix; Table S11 reports paired CPU/CUDA/Metal performance and concordance; and Table S15 maps each analysis to its source state and archive.

S1. Current implementation scope

Table S1. Stage-level CPU, CUDA, and Metal residency. ‘Hybrid’ denotes an accelerated path with one or more host-side operations.

Stage	CPU	CUDA	Metal	Residency / precision note
Core PLS fit	Compiled C++	GPU products/range; host small solve	MPS products; host direction update	CUDA/Metal hybrid; float32 smoke-tested
OPLS filter	Compiled C++	Host filter + CUDA core	Host filter + MPS core	Hybrid; float32 smoke-tested
Kernel PLS	Host kernel + C++ core	Host nonlinear Gram + CUDA core	Host nonlinear Gram + MPS core	Nonlinear K is $O(n^2)$; hybrid
Prediction	Compiled projection	CUDA projection; host	MPS projection; host post-	Transfers remain part of

Stage	CPU	CUDA	Metal	Residency / precision note
		model/result I/O	processing	execution
LDA	Compiled covariance/Cholesky	GPU moments/solve/score	MPS projection + host solve	float32 CPU/CUDA agreement verified
10-fold CV	Compiled folds/fits/scoring	Host scheduler + sequential GPU folds	Compiled/hybrid fold fits	No R-level refit loop; no concurrent folds

The double-precision CPU backend is the broadest reference implementation. Native float32 is selected automatically when input matrices are float::float32 objects and is available for route-specific PLS-SVD, SIMPLS, OPLS, and kernel PLS combinations on supported CPU, CUDA, and Metal builds. Windows now has a portable float-package CPU fallback for rSVD PLS-SVD, SIMPLS, and linear-kernel SIMPLS with argmax; it is not the native compiled single-precision path. Windows float32 OPLS, nonlinear kernel PLS, LDA, CUDA, and Metal combinations stop before fitting instead of silently converting to double.

Estimator identity is a dispatch invariant. A requested SIMPLS model is fitted with the SIMPLS core on CPU, CUDA, and Metal; the CUDA fused SIMPLS-LDA route supplies the SIMPLS method identifier to the native core and trains LDA from those SIMPLS scores. Label-aware PLS-SVD remains available only through an explicit PLS-SVD request. When the guarded CUDA SIMPLS response representation is too large, execution stops with an informative error instead of replacing the estimator. Benchmark code checks requested against executed estimator identifiers and records an estimator-mismatch error if they differ.

S2. Numerical algorithms

Notation follows the main text: a indexes the current latent component, A is the retained component count, K is the number of cross-validation folds, and r is the target rank of an SVD calculation. Lowercase k is used only for retrieval or evaluation cutoffs, such as nearest-neighbour neighbourhood size and top-k accuracy, and never for PLS component indexing. The public R argument ncomp supplies A.

S2.1 PLS-SVD

Let centred predictor and response matrices be $X \in \mathbb{R}^{n \times p}$ and $Y \in \mathbb{R}^{n \times q}$. PLS-SVD computes dominant singular directions of

$$S = X^{T} Y .$$

The decomposition is requested once at the maximum valid component count. Prefixes are reused to produce all component numbers requested by the user. The maximum effective rank is bounded by $min\{rank(X), rank(Y)\}$ and by the numerical rank of S. For a centred C-class indicator response, the columns sum to zero and $rank(Y) \leq C - 1$; the public factor-response path therefore caps PLS-SVD at $C - 1$ components. The model stores compact latent factors for prediction and omits a full coefficient cube when its estimated size exceeds the internal storage threshold and latent prediction is available.

Algorithm S1. PLS-SVD component-path construction.

1. Centre and, if requested, scale X with training statistics; centre numeric or indicator-coded Y.
2. Represent $S = X^{T} Y$ explicitly or as a product operator.
3. Compute the leading A singular triplets once, where A is the largest valid requested component count.
4. For each requested a $\in$ C, form the compact latent prediction factors from prefixes 1,...,a.
5. Store only the requested coefficient or prediction summaries; use blocked latent prediction when a dense coefficient path would exceed its memory threshold.

S2.2 SIMPLS

The SIMPLS estimator follows de Jong's sequential construction. For component a, the implementation obtains a dominant left direction r_a from the current cross-covariance state, forms and normalizes the score $t_a = X r_a$, computes predictor and response loadings, orthogonalizes the predictor loading against the preceding SIMPLS basis, and applies rank-one deflation. The default implementation accepts one refreshed direction after every deflation step.

The following execution optimizations are active in the current standard path:

1. Each randomized SIMPLS direction is generated from a fresh oversampled range sketch of the current deflated cross-covariance operator.
2. The row product $v_a^T S_a$ is evaluated once and reused in the rank-one deflation.
3. For tall matrices with moderate p, a shape-aware path caches $X^T X$ and the initial cross-covariance so that score normalization and loadings avoid repeated observation-space products.
4. The coefficient matrix is updated as $B_a = B_{a-1} + r_a q_a^T$.
5. When fitted values are requested, they are updated as $\hat{Y}_a = \hat{Y}_{a-1} + t_a q_a^T$.
6. Several requested component counts are extracted from one maximal path.

Incremental deflation, coefficient, and prediction updates reorganize numerical work while retaining sequential SIMPLS orthogonalization and deflation. The former one-vector warm-start and adaptive randomized refresh were rejected by formal validation and are not used by the public algorithm.

Algorithm S2. Accelerated sequential SIMPLS.

1. Centre and optionally scale X and centre Y; initialize $S = X^{\top} Y$, empty bases R, Q, and V, and the coefficient accumulator $B = 0$.
2. For component a, obtain one dominant left direction of the current deflated cross-covariance operator. IRLBA performs a fresh deterministic iterative solve. rSVD forms a fresh Gaussian sketch with target width one plus oversampling, applies the requested power iterations, performs QR orthonormalization, and extracts the leading vector from the reduced decomposition.
3. Form $t_a = X r_a$, normalize t_a and r_a, and calculate $p_a = X^{\top} t_a$ and $q_a = Y^{\top} t_a$.
4. Orthogonalize p_a against the existing V basis, reorthogonalize once, and normalize the resulting v_a.
5. Evaluate $h_a = v_a^{\top} S_a$ once and update $S_{a+1} = S_a - v_a h_a$.
6. Append r_a, q_a, and v_a; update $B_a = B_{a-1} + r_a q_a^{\top}$ and, when requested, $\hat{Y}_a = \hat{Y}_{a-1} + t_a q_a^{\top}$.
7. Snapshot outputs only at requested component counts. The optional $X^{\top} X$ cache changes how score norms and loadings are evaluated, not the quantities defined above.

Table S2. Dominant computational and storage terms. Here, A is the retained number of PLS components and l is the randomized-SVD sketch width (requested rank plus oversampling). Exact constants depend on matrix shape, solver convergence, and backend.

Path	Dominant work	Additional large storage	Memory-saving mechanism
Explicit PLS-SVD	Form S = X^TY: O(npq), then one truncated decomposition	S: O(pq); latent factors O((p+q)a)	One decomposition supplies every requested component prefix
Implicit PLS-SVD	For sketch width l, SΩ = $X^T(Y\Omega)$ and $S^TQ = Y^T(XQ)$	Sketches/factors O((p+q)l); no S	Matrix-free products; label-aware class sums avoid dense indicator Y
SIMPLS	a sequential leading-direction solves plus score/loading products	Explicit S: O(pq); bases O((p+q)a)	Cached deflation products and incremental prediction path; rSVD reuse is explicitly approximate
OPLS	Explicit S = X^TY and one-vector filter SVD, then SIMPLS predictive core	Filter S: O(pq); bases O(p·north) plus inner-model storage	No dense deflated X copy; the current orthogonal-filter stage remains explicit
Nonlinear kernel PLS	Kernel construction plus SIMPLS in sample space	Centred Gram matrix $O(n^2)$	Blocked test prediction only; nonlinear kernel storage remains quadratic

Path	Dominant work	Additional large storage	Memory-saving mechanism
Compact prediction	XtestR followed by latent response mapping	Test-score block and latent factors, not all p×q coefficient prefixes	Row-block streaming and low-rank latent mapping

S2.3 OPLS

OPLS estimates orthogonal scores and loadings from X, removes variation that is orthogonal to the response-associated direction, and fits the predictive SIMPLS core to the filtered matrix. Prediction applies the stored training filter before the predictive model. The current CUDA path performs the OPLS filter on the CPU and the predictive SIMPLS stage on CUDA. Metal accelerates the larger filter products but retains selected host-side operations.

S2.4 Kernel PLS

The linear kernel dispatches to the ordinary linear SIMPLS route, avoiding an unnecessary n-by-n Gram matrix. RBF and polynomial kernels construct and centre an explicit training Gram matrix and apply stored training means to held-out kernels. One float64 Gram matrix requires 8n^2 bytes before copies and workspaces: 0.75 GiB at n=10,000, 2.98 GiB at n=20,000, 6.71 GiB at n=30,000, and 18.63 GiB at n=50,000. Kernel centring, temporary products, and the R process require additional memory. The deterministic nonlinear-kernel validation included n_train=42-180; it establishes numerical agreement in that range, not large-sample feasibility. Conservative planning limits for the tested hardware are n<=20,000 on the 32-GB workstation and n<=10,000 on the 8-GB unified-memory Mac. These are operational ceilings based on reserving memory for multiple dense buffers, not measured performance limits. Larger nonlinear-kernel jobs were not validated and require an explicit memory calculation. The current CUDA nonlinear-kernel route is host assisted, so the 16-GB device does not remove the host-memory constraint.

S2.5 Randomized SVD

For a linear operator M and target rank r, rSVD draws a Gaussian matrix, forms a range sketch, optionally applies alternating products with M and its transpose, orthonormalizes the range, and decomposes the reduced matrix. The public one-power setting is intended for exploratory speed. It must not be interpreted as certified agreement; power, oversampling, and seed are recorded in every benchmark manifest.

CUDA and Metal offload the large matrix products, but neither backend should be described as universally device-resident. In the current float32 CUDA implementation, QR and the reduced decomposition are finalized in host float32 after GPU range products. Metal likewise retains selected host-side reduced operations and SIMPLS direction updates. Supplementary Table S1 records residency by stage rather than assigning a single label to an entire model family.

S2.6 Bundled IRLBA

The double CPU route bundles augmented implicitly restarted Lanczos bidiagonalization. It expands and restarts a bidiagonal Krylov subspace to approximate the requested dominant singular triplets without a full decomposition. The default public controls are maxit=1000, tol=1e-5, eps=1e-9, and svtol=1e-5, unless overridden through .... Exact fallback is used when either input dimension is below six. IRLBA is otherwise retained even when the requested rank approaches the smaller matrix dimension. The native float32 CPU and Metal routes use separate single-precision IRLBA-style implementations; CUDA float32 currently supports rSVD only.

S2.7 Implicit cross-covariance products

When $S = X^{\top} Y$ is expensive to store, the SVD iteration can apply the same operator through

$$S V = X^{\top}(Y V), S^{\top} U = Y^{\top}(X U).$$

For class labels $y_i \in \{1,\ldots,C\}$, multiplication by one-hot Y is replaced by indexing, and multiplication by Y^T is replaced by class-wise sums. With the centred indicator response, the explicit class-sum identity is

$$S_{:c} = \sum_{i:\, y_i = c} x_i - \frac{n_c}{n} \sum_{i=1}^{n} x_i .$$

The same centring and scaling values are applied in every row block. No independent SVD is fitted to separate blocks, so the mathematical operator remains the full cross-covariance.

The package uses distinct internal activation rules for rSVD and IRLBA because their iteration and memory profiles differ. These rules are implementation controls, not user-facing statistical parameters.

S2.8 Compact and streamed prediction

For SIMPLS, a prediction at retained component count A is evaluated through compact latent factors rather than a dense coefficient matrix. Large test matrices are processed in row blocks and intermediate score matrices are released after use. This limits retained prediction state to the current block and low-rank factors; it does not introduce a separate SVD solver.

S3. Classification heads

S3.1 Argmax

Argmax applies PLS regression to the multivariate class response and selects the class with the largest predicted response score. It is the standard PLS-DA decoding rule and requires no separate classifier fit.

S3.2 Latent-space LDA

LDA is fitted to the PLS training scores. For class counts n_c and means μ_c, the pooled covariance is computed from moments:

$$\Sigma = \frac{T^{\top} T - \sum_{c} n_c \mu_c \mu_c^{\top}}{max(1, n - C)}.$$

For each class, the system $\Sigma w_c = \mu_c$ is solved by Cholesky factorization and triangular solves. If factorization fails, the implementation sets $s = tr(\Sigma)/d$ when finite and positive, or $s = 1$ otherwise, and tries $\Sigma + \rho s I$ for $\rho \in \{10^{-8}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ in this fixed order. The sequence is a numerical fallback, not a tuned hyperparameter. The discriminant is

$$\delta_c(t) = t^{\top} w_c - \frac{1}{2} \mu_c^{\top} w_c + \log(n_c / n).$$

The float32 CPU, CUDA, and Metal routes keep their supported PLS arithmetic in the input precision. CUDA LDA uses single-precision score, covariance, factorization, and prediction buffers; Metal presently uses accelerated score projection with host-side LDA factorization. A controlled CPU/CUDA/Metal screen verified identical decoded labels for all four PLS families on a fixed classification task and relative prediction differences no larger than 2.4×10−6 on a fixed univariate-regression task.

S4. Cross-validation and metrics

pls.single.cv() evaluates eligible combinations of component count and prediction settings within one K-fold cross-validation layer. pls.double.cv() adds an outer layer for unbiased performance estimation. Selection can use accuracy, balanced accuracy, R^2, Q^2, or RMSD. Balanced accuracy is the unweighted mean of class-specific recalls and is intended for unequal class frequencies. When a nested permutation test is requested, the sampled null distribution uses the same selection endpoint and the appropriate upper or lower tail. The constrain input assigns related samples, such as repeated measurements from one patient, to the same fold; leave-one-group-out behaviour therefore never separates related observations.

Two uses of component grids are distinguished. Benchmark trajectory figures evaluate prespecified component counts on a fixed test set and are descriptive. Training-only model selection reports the best value within the evaluated grid. Endpoint selections are treated as censored with respect to the unobserved continuation of the prediction curve. Values at the lower or upper tested boundary, and PLS-SVD values constrained by response rank, are labelled explicitly and are never called optima. pls.single.cv() supports training-set selection, whereas pls.double.cv() provides an outer held-out layer when an unbiased estimate after tuning is required.

For observed labels y_i and decoded predictions $\hat{y}_i$, accuracy is $n^{-1}\sum_i I\left(y_i = \hat{y}_i\right)$. Top-$k$ accuracy replaces this indicator with membership of y_i among the k highest class scores. For a numeric response, training fit is summarized by

$$R^2 = 1 - \frac{\sum_{i,j}\left(y_{ij} - \hat{y}_{ij}\right)^2}{\sum_{i,j}\left(y_{ij} - \acute{y}_j\right)^2},$$

whereas cross-validated prediction is summarized by

$$Q^2 = 1 - \frac{\sum_{i,j}\left(y_{ij} - \hat{y}_{ij}^{(-f(i))}\right)^2}{\sum_{i,j}\left(y_{ij} - \acute{y}_j^{(-f(i))}\right)^2}.$$

Here $f(i)$ is the held-out fold and each mean is estimated without the held-out observation. Multivariate RMSD is $\left\{(nq)^{-1}\sum_{i,j}\left(y_{ij} - \hat{y}_{ij}\right)^2\right\}^{1/2}$, i.e. one global error over all response entries. For PLS-DA, R^2 and Q^2 use the centred indicator response, while accuracy evaluates decoded labels. Fitted-value R^2 and held-out Q^2 are therefore not interchangeable.

S5. Real datasets and preprocessing

The benchmark writes a machine-generated dataset manifest containing source, licence, checksum, object names, preprocessing, split seed, n_{train}, n_{test}, p, q, and component grid. Dataset dimensions are read from the prepared task objects rather than transcribed manually.

Table S3. Dataset dimensions represented by the current prepared benchmark objects. The released manifest is the authoritative machine-readable record and reports counts before and after each filtering stage. CBMC CITE-seq q=10 denotes the ten raw ADT count responses; its RMSD is reported in those assay-count units.

Dataset	Task	Prepared n train	Prepared n test	Prepared p	Prepared q
TCGA-HNSC methylation	Classification	520	58	782	2
CCLE	Classification	547	71	1,000	18
TCGA-BRCA	Classification	756	88	1,000	5
MetRef	Classification	773	100	375	22
TCGA Pan-Cancer	Classification	3,000	982	850	32
GTEx v8	Classification	3,000	797	1,000	32
Retina	Classification	22,402	22,406	50	12
Tabula Muris	Classification	50,043	50,059	50	32
CIFAR-100	Classification	50,000	10,000	768	100
PRISM	Multivariate regression	479	54	1,000	4,686
NMR	Multivariate regression	1,200	321	13,000	28,355
CBMC CITE-seq	Multivariate regression	7,755	862	1,000	10
ImageNet/DINOv2	Exploratory classification/retrieval stress test	1,000,000	281,167	1,024	1,000

S5.1 MetRef

MetRef comprises longitudinal urine proton-NMR spectra from 22 donors. Zero-signal variables are removed, spectra are normalized and scaled with training-derived statistics, and donor is the class label. The manifest identifies the normalization function and retained bins.

S5.2 CIFAR-100 and ImageNet

CIFAR-100 and ImageNet were evaluated as precomputed image embeddings rather than by training image encoders. CIFAR-100 used its standard 50,000/10,000 partition [30]. The ImageNet archive contained 1,281,167 rows, 1,024 DINOv2 features, and 1,000 labels [28,29], but no authoritative canonical train/validation flag. The feature archive was reformatted without normalization or dimensionality reduction. Its exact DINOv2 checkpoint and pooling rule were not retained in independently auditable metadata, which limits representation-level reproducibility and is stated explicitly. Feature extraction was completed before the fastPLS benchmark and excluded from timing.

ImageNet is included as a large-scale foundation-model embedding stress test, not as a biomedical validation set. This distinction is relevant because computational pathology foundation models such as UNI and Prov-GigaPath produce large collections of dense tile or slide representations for supervised downstream analysis [31,32]. The ImageNet experiment tests whether fastPLS can process the corresponding post-extraction matrix regime. It does not establish transfer of natural-image accuracy to pathology, and pathology-specific predictive claims require direct pathology data.

S5.3 GTEx, TCGA, and CCLE

GTEx v8 tissue labels, TCGA Pan-Cancer disease labels, TCGA-BRCA PAM50 subtypes, TCGA-HNSC sample type, and CCLE lineage labels define classification outcomes. Outcome eligibility filtering and sample matching occur before the split; predictor imputation, zero-variance filtering, normalization, scaling, and any data-derived feature selection are estimated from training data only. Dataset-specific minimum-class rules and the number of removed observations are recorded in the manifest. Legacy descriptions such as “the first 1,000 features” denote a target maximum rather than the final matrix width: matching, finite-value checks, and zero-variance removal can reduce p, explaining prepared widths such as 850 or 782. The ordered retained identifiers, not the source-file column positions alone, define the reproducible feature set.

S5.4 CBMC CITE-seq, Retina, and Tabula Muris

CBMC CITE-seq [21] used the prepared SeuratData CBMC object (dataset version 3.1.4; 8,617 matched cells). X was the stored untransformed integer RNA assay-count matrix restricted before benchmarking to 1,000 preselected genes. Y was the stored untransformed integer ADT assay-count matrix for CD3, CD4, CD8, CD45RA, CD56, CD16, CD11c, CD14, CD19, and CD34. The stored split contained 7,755 training and 862 held-out cells and was loaded unchanged. The benchmark loader applied no library-size normalization, log transformation, centred-log-ratio transformation, or additional feature selection. The PLS call used mean centering without variance scaling: training means were subtracted from X and Y, held-out X was centered with the training X means, and the training Y means were restored to predictions. Consequently, the reported global RMSD is the square root of the mean squared error across all 8,620 held-out response entries (862 cells x 10 markers), in original ADT assay-count units per cell. The marker scales are heterogeneous, so this pooled RMSD is weighted toward high-count markers and is not numerically comparable with RMSD measured on normalized responses in another dataset. For the selected CUDA SIMPLS result, the global RMSD of 1,056.461 combined marker-wise RMSE values from 56.4 for CD34 to 3,191.3 for CD45RA. Retina and Tabula Muris are separate single-cell classification benchmarks and are named separately throughout the released manifest.

S5.5 PRISM and NMR

PRISM matches molecular profiles of cancer cell lines to multivariate drug-response measurements. NMR predicts a high-dimensional response spectrum from the input spectral representation. Both are multivariate regression tasks and are evaluated by RMSD rather than classification accuracy. The NMR source cohort and the prepared benchmark object are distinct: Table S3 reports the 1,521 matched, analysis-ready observations in the prepared object, not the complete source cohort. The released manifest must enumerate exclusions and train/test allocation so that these counts can be reconciled without inference. The NMR data are associated with the study of Vignoli, Cacciatore, and Tenori; source-study ethics and access conditions follow the cited source study.

S5.6 Redistribution and executable acquisition

Redistribution status was determined separately from source access status. A publicly downloadable source was not treated as permission to redistribute the processed benchmark matrix under the fastPLS licence. Only the GPL-compatible breast and colon package examples, synthetic generators and generated synthetic results, and aggregate tables, figures, manifests, split indices, and checksums are redistributed. None of the prepared real-data matrices in Table S3 is bundled. The executable workflow benchmark/acquire_publication_datasets.R downloads authoritative public sources or validates user-supplied local files, writes paths, sizes, access classes, and checksums to acquisition_manifest.csv, and refuses silent release substitution. Exact commands and preprocessing contracts are documented in benchmark/DATA_ACQUISITION.md.

Table S3a. Redistribution and acquisition status. 'No' means that fastPLS does not redistribute the prepared real-data matrix, even when the upstream source is public. The acquisition command is run from the repository root; restricted validation requires the environment variable shown.

Dataset/object	Authoritative source	Access class	Prepared object redistributed?	Executable route
breast; colon	mixOmics; plsgenomics	GPL-compatible package data	Yes	data(breast); data(colon)
Synthetic and aggregate outputs	fastPLS generators/results	Project-generated	Yes	Run published benchmark scripts
MetRef	KODAMA::MetRef	Public R package	No	--dataset=metref
CIFAR-100	University of Toronto	Public download	No	--dataset=cifar100
CBMC CITE-seq	GEO GSE100866	Public repository	No	--dataset=cbmc_citeseq
Retina	GEO GSE63472; openTSNE pca_50	Public repositories	No	--dataset=retina
Tabula Muris	ExperimentHub EH1617	Public Bioconductor data	No	--dataset=tabula
GTEx v8	GTEx open access via UCSC Xena	Open expression/phenotype	No	--dataset=gtex_v8
TCGA tasks	TCGA open access via UCSC Xena	Open molecular/phenotype	No	--dataset=tcga_brca,tcga_hnsc_methylation,tcga_pan_cancer
CCLE	DepMap/CCLE 18Q2	Historical release	No	FASTPLS_CCLE_RDATA=... --dataset=ccle
PRISM	DepMap PRISM 19Q4	Release-controlled	No	FASTPLS_PRISM_RDATA=... --dataset=prism
NMR	Figshare share; MTBLS242/395/424	Source-study terms	No	FASTPLS_NMR_RDATA=... --dataset=nmr
ImageNet/DINOv2	ImageNet authorized copy; local extraction	Gated/non-commercial terms	No	FASTPLS_IMAGENET_RDATA=... --dataset=imagenet

The public acquisition command is Rscript benchmark/acquire_publication_datasets.R --dataset=<id> --out=<directory>. For ImageNet, NMR, CCLE, and PRISM, the listed environment variable points to a user-authorized local file. The workflow records a checksum but does not copy that file into the repository. GTEx uses only open-access expression and phenotype data; protected sequence and full donor-level files are outside the benchmark. The ImageNet embeddings remain a derived object subject to the source-image terms and are not redistributed.

S6. Synthetic datasets and corresponding numerical results

The simulated datasets were used only for the formal SIMPLS estimator-comparison and rSVD reliability analyses; no separate n/p/q performance-scaling sweep is claimed. Five multivariate regression regimes and three dummy-response classification regimes were generated with seeds 101, 202, and 303. Regression data used independent standard-normal latent scores and Gaussian predictor and response loading matrices. Predictor and response matrices were formed by multiplying the latent-score matrix by the transposed loading matrices and adding independent Gaussian noise with standard deviation 0.05. The regimes covered $p < n$, $p > n$, low- and high-rank Y, near-collinear predictors, and exact rank deficiency. Near-collinearity was created by making two loading columns linear combinations of the first loading column plus noise with standard deviation 1×10^{-7}; exact rank deficiency was created by duplicating ten predictor columns. Training and held-out rows were generated in one draw and separated before model fitting. The authoritative deterministic and approximate summaries are Tables S7 and S8; exact regime dimensions remain in the repository result archive.

Classification regimes used balanced shuffled class labels and Gaussian class-specific latent prototypes. Each latent observation was its class prototype plus Gaussian noise with standard deviation 0.45; predictors were obtained through a Gaussian loading matrix with additional noise of standard deviation 0.08, and responses were one-hot class indicators. The three regimes covered $p < n$ with a low-rank response, $p > n$ with a higher-rank response, and near-collinear predictors. Every compared implementation received identical matrices, splits, component grids, folds, and seeds.

The corresponding deterministic results comprised 117 component-level comparisons. All met the prespecified numerical tolerances. Within synthetic classification, the worst relative prediction and coefficient errors were 2.55×10^{-8} and 4.91×10^{-8}; within synthetic regression they were 1.09×10^{-5} and 1.13×10^{-5}. Maximum score-subspace angles were 2.09×10^{-6} degrees for classification and 0.00143 degrees for regression. Fixed 5-fold component selection agreed with pls::simpls.fit in all eight synthetic tasks. The authoritative deterministic summary is Table S7; complete task-level endpoints and curves are retained in the repository archive.

For approximate rSVD, oversampling 10 with one power iteration met the prespecified numerical tolerances in 101 of 117 component-level checks, whereas two power iterations met them in 117 of 117. The focused CUDA audit met the prespecified numerical tolerances at all evaluated points with either four power iterations at oversampling 10 or oversampling 20. The authoritative numerical audit is Table S8; full endpoints remain in the repository archive. These are solver-reliability results, not estimator-equivalence evidence.

S7. Benchmark execution and memory measurement

Each fit is run in an isolated R process. Total time is fitting plus prediction and excludes package installation, dataset acquisition and loading, package initialization, and metric calculation after prediction. The fit timer starts immediately before the public pls() call and stops after the fitted R model is returned. For CUDA routes, this interval includes host-side argument marshalling, host-to-device transfer of training matrices, device allocation, CUDA kernels and library calls, synchronization, and device-to-host transfer of model components exposed in the returned R object. The prediction timer starts immediately before predict() and stops after predictions or class scores are available in R. It includes host-to-device transfer of the test matrix and any model arrays not retained on the device, CUDA execution and synchronization, and device-to-host result transfer. Routes supporting resident workspaces reuse them; the benchmark does not add an artificial host round trip, but the public R model remains the interface between the separately timed fit and prediction calls. Therefore, reported CUDA time is end-to-end backend time rather than kernel-only time. After data and libraries are loaded, garbage collection is completed and host RSS and process-specific GPU memory are recorded immediately before

fitting. The R process then signals a synchronized external sampler, which polls host RSS and PID-matched nvidia-smi device memory every 0.02 s throughout fitting and prediction. /usr/bin/time records the absolute maximum RSS over the complete isolated process. Incremental host memory is the maximum of fit-window samples and post-fit/post-prediction snapshots minus the pre-fit baseline; incremental process-level GPU memory is the PID-specific sampled peak minus its pre-fit baseline. Negative differences are truncated at zero. CUDA contexts were not preinitialized, so a zero pre-fit GPU baseline does not remove context creation, CUDA-library handles, allocator pools, persistent device objects, data/model storage, or workspaces created by the measured call. The GPU increment therefore measures the end-to-end device footprint triggered by the workflow and is not an isolated algorithmic-workspace measurement. Isolating workspace would require CUDA-API instrumentation after persistent runtime state had been initialized, which was not performed.

Current benchmark workflows record repository state, benchmark-script checksum, package version, source-archive SHA-256, compiler, BLAS/LAPACK, thread settings, accelerator libraries, seeds, rSVD controls, and data/split identifiers. Table S15 maps each analysis to its exact evidence archive. NMR and ImageNet used fastPLS_0.99.6.tar.gz (SHA-256 c868770fee196af24bfb15b4da9fad1d7765deaf68c0ef1fd3e5a15670ecaa85) from base commit 6e50bd318f20289101f6b723953830aefa8b95d6. The audited 0.99.10 archive (SHA-256 163ac7bd5c0c241f3817fac989e219f71b3956b388f6fcefa2f3420c45051b25) changes installation, API cleanup, documentation, and metric-selection behavior; benchmark values are not relabelled as 0.99.10 reruns.

Table S4a. Exact reproducibility environment for the primary CUDA workstation and the Apple Metal validation system. Values were read from benchmark manifests, sessionInfo(), R configuration, package installation logs, linked runtime libraries, and operating-system queries. The CUDA and Metal columns are not direct hardware comparisons.

Item	CUDA workstation	Metal validation system
Platform	Intel Core i7-13700 (16 cores, 24 logical CPUs); 32,526,480 kB RAM; NVIDIA GeForce RTX 5060 Ti, 16,311 MiB VRAM	macOS 14.5 (23F79); Apple M3 (8 CPU cores, 10 GPU cores); 8,589,934,592 bytes unified memory; Metal 3

Item	CUDA workstation	Metal validation system
R and fastPLS	R 4.6.0 and R 4.5.1 across campaigns; fastPLS 0.99.6. See Table S15 for analysis-specific source provenance.	R 4.6.0; fastPLS 0.99.6; package commit recorded by each Metal run. See Table S15.
Core R packages	Rcpp 1.1.1-1.1; RcppArmadillo 15.2.7-1; RcppEigen 0.3.4.0.2; Matrix 1.7-5; float 0.3-3; pls 2.9-0	Rcpp 1.1.2; RcppArmadillo 15.4.0-1; RcppEigen 0.3.4.0.2; Matrix 1.7-5; float 0.3-3; pls 2.9-0
External comparison packages	mdatools 0.15.0; chemometrics 1.4.4; pcv 1.1.0; plsdepot 0.3.1; plsgenomics 1.5-3; mixOmics 6.36.0; spls 2.3-2; ropls 1.44.0	Not used for the external-package comparison
C/C++ compiler	GCC/G++ 11.4.0; C++17	Homebrew Clang/Clang++ 22.1.1; C++17; macOS SDK 15.2
Effective C/C++ flags	-std=gnu++17 -fpic -g -O2 -ffile-prefix-map=/build/r-base-gN72Ro/r-base-4.6.0=. -fstack-protector-strong -Wformat -Werror=format-security -Wdate-time -D_FORTIFY_SOURCE=2; package definitions: -DRANN -DARMA_64BIT_WORD=1 -DFASTPLS_HAS_CUDA -DFASTPLS_HAS_CUDA_KERNELS	-std=gnu++17 -fPIC -falign-functions=64 -Wall -g -O2; Objective-C++: -x objective-c++ -fobjc-arc; package definitions: -DRANN -DARMA_64BIT_WORD=1 -DFASTPLS_HAS_Metal
Accelerator build/link	NVCC 13.0.88: -std=c++17 --extended-lambda --expt-relaxed-constexpr -DFASTPLS_HAS_CUDA -DFASTPLS_HAS_CUDA_KERNELS -Xcompiler -fPIC; linked with -lcudart -lcublas -lcusolver -lcurand	Linked frameworks: Foundation, Metal, and MetalPerformanceShaders
BLAS/LAPACK	Reference BLAS 3.10.0; LAPACK 3.10.0	R BLAS (/Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib); R LAPACK 3.12.1 (libRlapack.dylib)
CPU thread setting	1 effective BLAS thread; reference BLAS and no OpenMP; OMP_NUM_THREADS, OPENBLAS_NUM_THREADS, and MKL_NUM_THREADS unset	1 effective R-BLAS thread; no OpenMP; OMP_NUM_THREADS and VECLIB_MAXIMUM_THREADS unset
GPU runtime/libraries	NVIDIA driver 595.71.05; CUDA SDK 13.0.3; CUDA runtime 13.0.96; cuBLAS 13.1.1.3; cuSOLVER 12.0.4.66; cuRAND 10.4.0.35	Metal 3; Metal.framework and MetalPerformanceShaders.framework from macOS SDK 15.2

Table S4b. External implementation scope. Exact package versions and execution errors are captured by the run manifest.

Package	Functions evaluated where compatible	Comparison scope
pls	plsr(method="simpls"), simpls.fit	SIMPLS reference for supported regression or dummy-response tasks
mdatools	pls, plsda	PLS regression or PLS-DA when the installed API supports the task
plsdepot	simpls	Independent SIMPLS fit
pcv	simpls	Independent SIMPLS fit where response shape is supported
plsgenomics	pls.regression	Genomics-oriented PLS regression/classification where installable
chemometrics	pls_eigen, pls2_nipals	Eigen- and NIPALS-based PLS references
mixOmics	plsda, splsda	PLS-DA and sparse PLS-DA; model differences are labelled

Package	Functions evaluated where compatible	Comparison scope
spls	spls, splsda	Sparse estimators; not treated as identical to dense SIMPLS
ropls	opls	PLS/OPLS where task type and class-count restrictions permit

Only estimator/task combinations with compatible outputs are used for speed-and-prediction comparisons. Estimator-matched tables require the same preprocessing, response representation, component count, prediction head, precision, thread setting, and timed work. A separate complete-workflow table may compare different estimators or heads, but is not interpreted as an implementation-only speed test. Unsupported multivariate responses, binary-only restrictions, timeouts, memory kills, and numerical failures remain in the final table rather than being dropped. Adapter errors are corrected before comparison or labelled as benchmark-code failures; they are not described as limitations of the external package.

S8. Backend reproducibility

Reproducibility experiments use identical data, preprocessing, folds, requested components, and seeds for the configured CPU, CUDA, and Metal routes. CPU builds delegate dense linear algebra to the BLAS/LAPACK implementation linked to R and can use OpenBLAS where it is installed. That build capability is recorded separately from measured performance. The primary software-comparison runs used one effective BLAS thread. No controlled one-thread versus multiple-thread scaling experiment was conducted, and no multicore speed-up is inferred. Because rSVD is stochastic and nearly tied singular values can rotate an equivalent basis, raw component columns are sign-aligned only for descriptive comparisons. Primary endpoints are prediction agreement, absolute predictive-metric difference, relative Frobenius prediction error, selected component count, numerical failures, and run-to-run variability. Principal angles compare latent subspaces.

Table S5. Backend reproducibility endpoints.

Endpoint	Definition	Interpretation
Prediction agreement	Fraction of identical decoded labels, or relative Frobenius difference for numeric predictions	Primary check of model-output agreement under backend changes
Predictive-metric difference	Absolute difference in accuracy, Q^2, or RMSD	Statistical rather than bitwise equivalence
Principal angles	Angles between fitted latent subspaces	Robust to sign changes and rotations of nearly tied singular vectors
Selected-component agreement	Equality of the component count chosen on fixed folds	Checks downstream reproducibility of tuning
Numerical failures	Non-convergence, Cholesky fallback, non-finite output, timeout, or memory failure	Retained explicitly rather than excluded

The completed fixed-score LDA validation used MetRef, CIFAR-100, Retina, and Tabula Muris. CPU and CUDA float32 predictions were compared at fixed component counts; classifier-level agreement is reported separately from upstream rSVD variation.

S9. ImageNet computational role

The scientific role of this experiment is computational. ImageNet supplies a large, public, multiclass embedding matrix with the same broad data form as the tile-embedding matrices generated by modern computational pathology foundation models [31,32]. It therefore tests the feasibility, memory behaviour, and prediction throughput of PLS after feature extraction. It does not substitute for pathology validation, and no ImageNet result is interpreted as evidence of diagnostic performance.

S10. Reporting conventions

Completed, failed, timed-out, memory-killed, and unavailable routes remain explicit. Component counts at a tested boundary are described as best within the evaluated grid. Deterministic estimator validation is not pooled with approximate rSVD agreement, and speed-up is summarized only for numerically concordant paired routes.

S11. Authoritative evidence map

This compact supplement contains one authoritative table for each central evidence class. Expanded component paths, intermediate review-cycle summaries, sensitivity analyses, and diagnostic figures remain available through benchmark/MANUSCRIPT_EVIDENCE_ARCHIVE.md. They are reproducibility records, not parallel sources for manuscript claims.

Table S6. Claim-to-evidence map. Each central main-text claim is assigned to one authoritative Supplementary source.

Main-text claim		Authority	Evidence scope
Deterministic SIMPLS meets the prespecified numerical tolerances against de Jong SIMPLS		Table S7	117 component-level endpoints and 12 fixed-fold selections
OPLS and kernel-PLS implementation reliability		Table S7	66 setting/task endpoints, 66 selections, 1,540 fold-component fits
rSVD is approximate and requires numerical qualification		Table S8	CPU and CUDA oversampling/power-iteration audits
Backend stages are native, hybrid, or host-resident		Table S1	Stage-level CPU, CUDA, and Metal residency
float32 support is route conditional		Table S9	Family/backend/endpoint capability and warning policy
fastPLS versus independent R implementations		Table S10	Matched float64 one-CPU SIMPLS workflows
Selected-point predictive, time, and memory performance		Table S11	Paired CPU/CUDA dataset-family rows
NMR predictive selection and backend attribution are separate		Table S12	Family-selected errors, paired backends, historical workflow
ImageNet feasibility and supervised compression are exploratory		Table S13	Component path and matched FAISS representation study
Predictive dispersion includes training-sample variation		Table S14	Repeated outer partitions and selection frequencies
Each analysis maps to a reproducible archive		Table S15	Script, package metadata, split provenance, and archive digest
SIMPLS speed and storage gains arise from specific execution changes	Table S7a		Five same-code ablations on four matrix regimes
SIMPLS runtime relative to one-shot PLS-SVD is shape dependent	Table S7b		Five matched synthetic shapes on CPU and CUDA

S12. Deterministic estimator validation

Deterministic estimator comparison and approximate-solver agreement are separate questions. Table S7 contains only deterministic float64 CPU validation. rSVD is excluded and evaluated in Section S13.

Table S7. Definitive deterministic estimator-validation summary. Errors are worst observed values; angles are in degrees.

fastPLS scope	Reference	Endpoint tolerance status	Selection agreement	Max pred. rel. error	Max coef. rel. error	Max angle	Max metric diff.	Conclusion
SIMPLS / deterministic IRLBA	pls::simpls.fit / de Jong	117 / 117	24 / 24	1.09e-05	1.13e-05	0.0015	1.92e-06	Met deterministic tolerances
OPLS (north 1-3) and kernel PLS (8 settings)	Independent filter/kernel + pls::simpls.fit	66 / 66	66 / 66	9.48e-12	3.07e-10	2.41e-06	3.19e-12	Reliable in deterministic float64 CPU scope

S12.1 Execution ablation and direct PLS-family timing

Each ablation changed one internal execution feature while keeping the data, split, SIMPLS estimator, deterministic IRLBA solver, component count, seed, and prediction head fixed. Three isolated runs were used per configuration. Speed-up is reference time divided by optimized time; positive RSS reduction denotes a lower fit-window incremental peak. Results show that memory reduction and speed are separate and matrix-shape dependent.

Table S7a. Same-code SIMPLS execution ablation. All endpoint metric differences were zero and minimum classification agreement was 1.000.

Optimization	Datasets	Time speed-up range	Incremental RSS reduction	Interpretation
Cached deflation products	4	0.94-1.02x	0.16-70.60%	Memory benefit; no uniform time gain
Cached X-transpose-X	4 (active in 1)	0.99-1.00x	-0.06-0.57%	Shape-gated; negligible in this panel
Compact prediction	4	0.97-1.24x	-0.02-77.71%	Largest benefit for high-response PRISM
Incremental coefficients	4	0.99-1.07x	-3.66-2.68%	Small isolated effect
Implicit cross-covariance	4	0.065-6.24x	0.12-70.64%	Faster only in the high-response regime

The matched PLS-family timing study used the same synthetic matrices, requested components, rSVD controls (oversampling 10, one power iteration, seeds 101-103), split, and backend for PLS-SVD and SIMPLS. The two estimators can differ predictively; this experiment compares execution time only.

Table S7b. Direct matched-shape runtime comparison. Ratios are SIMPLS time divided by PLS-SVD time; values below one favor SIMPLS.

Shape	n / p / q / A	CPU ratio	CUDA ratio	Interpretation
Wide	400 / 2000 / 20 / 10	1.18	0.93	Near parity
Tall-thin	5000 / 50 / 20 / 10	1.00	0.94	Near parity
High response	1000 / 300 / 500 / 50	1.11	0.94	Near parity
Balanced	5000 / 500 / 50 / 50	3.84	0.98	CPU favors PLS-SVD
High components	3000 / 768 / 200 / 100	1.34	0.92	CUDA near parity

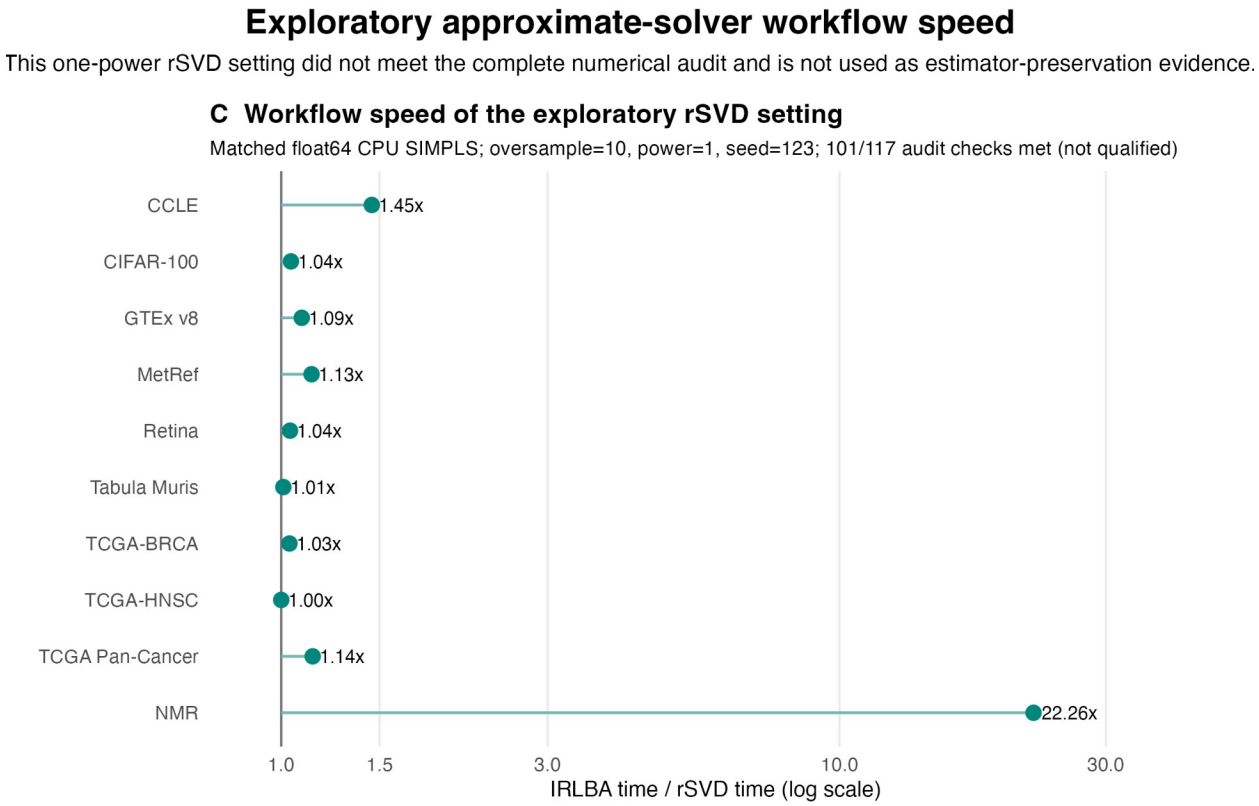
S13. Approximate rSVD reliability

rSVD uses a fixed seed, Gaussian range sketch, oversampling, and power iterations. Qualification required all of the following: relative Frobenius prediction error <= 0.05, prediction correlation >= 0.99, each score/projection/loading subspace angle <= 10 degrees, decoded-label agreement >= 0.99, and absolute predictive-metric difference <= 0.01. These prespecified values were computational non-inferiority screens rather than clinical or estimator-equivalence margins. The relative error is scale normalized but can conceal localized errors, so it was never interpreted without the endpoint-specific metric and, for classification, exact discordant counts. An agreement threshold of 0.99 permits no changed prediction for test sets smaller than 100 and at most one for n = 100. In the qualified CPU two-power analysis, the maximum relative error of 0.0332 occurred on MetRef at 18 components with zero label or accuracy change; the three 0.99-agreement endpoints each contained one discordant MetRef prediction among 100 and changed accuracy by zero or 0.01. The largest regression metric difference was 0.000693. Thus, qualification supports approximate computational use, not a claim that individual predictions are interchangeable. Deterministic IRLBA is recommended for confirmatory small-cohort or case-level inference.

Table S8. Definitive rSVD numerical-audit summary. CPU contains 117 component-level tests; CUDA contains eight task/endpoint tests per setting.

Backend	Oversample	Power	Tolerance checks met	Max pred. rel. error	Min pred. corr.	Max score angle	Min label agree.	Max metric diff.	Status
CPU	10	1	101/117	0.2398	0.97173	25.28	0.930	0.0400	Rejected
CPU	10	2	117/117	0.0332	0.99939	4.93	0.990	0.0100	Qualified approximate
CUDA	10	1	0/8	0.2486	0.96884	NR	0.830	0.0300	Rejected
CUDA	10	2	5/8	0.0822	0.99602	NR	0.940	0.0200	Rejected
CUDA	10	4	8/8	0.0078	0.99996	NR	0.990	6.75e-05	Qualified approximate
CUDA	20	1	8/8	0.0027	1.00000	NR	1.000	0.0002	Qualified approximate
CUDA	20	2	8/8	6.26e-06	1.00000	NR	1.000	3.78e-07	Qualified approximate
CUDA	20	4	8/8	6.20e-06	1.00000	NR	1.000	1.05e-07	Qualified approximate

Figure S1. Exploratory one-power rSVD workflow speed relative to deterministic IRLBA. The setting used oversampling 10, one power iteration, and seed 123; it met only 101/117 audit checks and is excluded from estimator-preservation claims.



S14. float32 capability

float64 is the confirmatory reference. float32 input halves raw matrix storage but does not guarantee lower peak process memory, faster execution, or numerical agreement. Public calls allow validated routes, warn for experimental, hybrid, or measured-risk routes, and stop before allocation for unavailable routes.

Table S9. Definitive float32 capability matrix. Endpoint statuses are validated, experimental, hybrid, unavailable, or failed; extreme-response status refers to q >= 10,000 with at least 50 components.

Family	Kernel	Backend	Endpoint status	Solver	Residency	Windows	Extreme q
PLS-SVD	n/a	CPU	regression=validated; argmax=validated; LDA=validated	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	experimental/not applicable
PLS-SVD	n/a	CUDA	regression=validated; argmax=validated; LDA=validated	rSVD only	device	unavailable	experimental/not applicable
PLS-SVD	n/a	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	experimental/not applicable
SIMPLS	n/a	CPU	regression=validated; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	failed/not applicable
SIMPLS	n/a	CUDA	regression=validated; argmax=experimental; LDA=experimental	rSVD only	Hybrid in float32 label-aware classification: host sequential SIMPLS and score projection; CUDA rSVD range products and LDA	unavailable	failed/not applicable
SIMPLS	n/a	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	failed/not applicable
OPLS	n/a	CPU	regression=validated; argmax=validated; LDA=validated	rSVD validated; IRLBA experimental	CPU	unavailable	failed/not applicable
OPLS	n/a	CUDA	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD only	CUDA inner PLS with host-resident OPLS orchestration	unavailable	failed/not applicable
OPLS	n/a	Metal	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD and IRLBA experimental	Metal inner PLS with host-resident OPLS orchestration	unavailable	failed/not applicable
kernel PLS	linear	CPU	regression=validated; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU	portable CPU rSVD regression/argmax; native LDA unavailable	failed/not applicable
kernel PLS	linear	CUDA	regression=validated; argmax=experimental; LDA=experimental	rSVD only	device	unavailable	failed/not applicable

Family	Kernel	Backend	Endpoint status	Solver	Residency	Windows	Extreme q
kernel PLS	linear	Metal	regression=experimental; argmax=experimental; LDA=hybrid	rSVD and IRLBA experimental	Metal PLS scores plus CPU float32 LDA/device with hybrid reduced decomposition	unavailable	failed/not applicable
kernel PLS	nonlinear	CPU	regression=experimental; argmax=experimental; LDA=experimental	rSVD validated; IRLBA experimental	CPU with explicit Gram matrix	unavailable	failed/not applicable
kernel PLS	nonlinear	CUDA	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD only	CUDA kernel products with host-resident Gram centering/orchestration	unavailable	failed/not applicable
kernel PLS	nonlinear	Metal	regression=hybrid; argmax=hybrid; LDA=hybrid	rSVD and IRLBA experimental	Metal kernel products with host-resident Gram centering/orchestration	unavailable	failed/not applicable

S15. External software comparison

The primary software comparison used float64, deterministic CPU SIMPLS, the same split and component count, and one effective BLAS thread. fastPLS argmax is estimator matched to pls::simpls.fit; LDA is a workflow comparison because the prediction head differs. Memory is absolute process RSS and is reported for feasibility, not isolated algorithmic allocation. Across 126 attempted external-package dataset/method runs, 110 completed and 16 did not: 12 were documented package limitations, two were killed at the timeout, and two produced execution errors. The previously incomplete CIFAR-100 fastPLS SIMPLS-LDA row was rerun independently with a 7,200-s limit; all three replicates completed, with median accuracy 0.8710, median total time 10.118 s, and median peak process RSS 1,687 MB. The current package-comparison workflow uses a 10,000-s default timeout. The isolated evidence is stored under benchmark_results/manuscript_revision_cycle78_20260726/cifar100_fastpls_simpls_lda.

Table S10. Definitive one-CPU external comparison. Cells report accuracy; total fitting-plus-prediction time; peak process RSS.

Dataset	A	fastPLS argmax	fastPLS LDA	Fastest external	Best-accuracy external
CCLE	50	0.7324; 0.077s; 469MB	0.7746; 0.091s; 470MB	pls/SIMPLS: 0.7324; 0.109s; 135MB	plsgenomics/PLS-LDA: 0.7746; 0.114s; 172MB
GTEx v8	32	0.9310; 0.234s; 509MB	0.9611; 0.274s; 498MB	plsgenomics/PLS-LDA: 0.9611; 0.360s; 203MB	plsgenomics/PLS-LDA: 0.9611; 0.360s; 203MB
MetRef	22	0.7700; 0.034s; 470MB	0.8800; 0.037s; 470MB	pls/SIMPLS: 0.7700; 0.037s; 125MB	plsgenomics/PLS-LDA: 0.8800; 0.038s; 146MB
Retina	50	0.9678; 0.084s; 530MB	0.9691; 0.118s; 517MB	chemometrics/PLS eigen: 0.9344; 0.136s; 163MB	plsgenomics/PLS-LDA: 0.9691; 0.760s; 289MB
Tabula Muris	50	0.8006; 0.346s; 682MB	0.8752; 0.309s; 596MB	chemometrics/PLS eigen: 0.7833; 0.605s; 314MB	plsgenomics/PLS-LDA: 0.8752; 2.092s; 492MB
TCGA- BRCA	5	0.8523; 0.030s; 471MB	0.8750; 0.032s; 470MB	plsgenomics/PLS-LDA: 0.8750; 0.021s; 145MB	chemometrics/PLS eigen: 0.8864; 1.036s; 180MB
TCGA-HNSC methyl.	2	1.0000; 0.023s; 467MB	0.9828; 0.023s; 467MB	plsgenomics/PLS-LDA: 0.9828; 0.010s; 133MB	pls/SIMPLS: 1.0000; 0.013s; 107MB
TCGA Pan- Cancer	50	0.9114; 0.263s; 500MB	0.9155; 0.323s; 495MB	plsgenomics/PLS-LDA: 0.9155; 0.533s; 221MB	plsgenomics/PLS-LDA: 0.9155; 0.533s; 221MB
CIFAR-100	100	0.8739; 8.189s; 1688MB	0.8710; 10.118s; 1687MB	pls/SIMPLS: 0.8739; 34.647s; 13090MB	pls/SIMPLS: 0.8739; 34.647s; 13090MB

S16. Selected-point CPU and CUDA benchmark

Every row uses the component count selected from the training data for that dataset and family. CPU and CUDA values share the split, model family, precision, solver family, and requested component count. Incremental host RSS is the fit-window peak minus the immediately pre-fit baseline. Incremental GPU memory includes runtime/context state and is not workspace-only allocation.

Table S11. Definitive selected-point performance. Metric, time, and host memory are CPU / CUDA; GPU memory is the CUDA increment. Endpoint choices are best within the evaluated training grid.

Dataset	Family	A	Selection status	Metric CPU / CUDA	Time s CPU / CUDA	Delta host MB CPU / CUDA	Delta GPU MB	Numerical status
metref	kernel PLS	100	upper boundary	Acc 0.9800 / Acc 0.9800	0.044 / 0.240	18.2 / 314.2	198.0	metric concordant
metref	OPLS	100	upper boundary	Acc 0.9800 / Acc 0.9800	0.050 / 0.246	10.6 / 317.7	198.0	metric concordant
metref	PLS-SVD	21	response-rank limit	Acc 0.8000 / Acc 0.8000	0.015 / 0.239	4.2 / 389.7	198.0	metric concordant
metref	SIMPLS	100	upper boundary	Acc 0.9800 / Acc 0.9800	0.043 / 0.240	6.5 / 311.0	198.0	metric concordant
ccle	kernel PLS	18	interior tested value	Acc 0.6620 / Acc 0.6479	0.026 / 0.211	4.6 / 331.2	200.0	metric discordant
ccle	OPLS	18	interior tested value	Acc 0.6056 / Acc 0.6056	0.037 / 0.220	14.4 / 331.3	200.0	metric concordant
ccle	PLS-SVD	17	response-rank limit	Acc 0.6479 / Acc 0.6479	0.020 / 0.237	2.4 / 374.9	200.0	metric concordant
ccle	SIMPLS	18	interior tested value	Acc 0.6620 / Acc 0.6479	0.026 / 0.218	5.1 / 354.5	200.0	metric discordant
tcga brca	kernel PLS	10	interior tested value	Acc 0.8523 / Acc 0.8523	0.023 / 0.224	5.8 / 392.4	200.0	metric concordant
tcga brca	OPLS	5	interior tested value	Acc 0.8068 / Acc 0.7841	0.025 / 0.223	19.0 / 344.5	200.0	metric discordant
tcga brca	PLS-SVD	4	interior tested value	Acc 0.8864 / Acc 0.8864	0.014 / 0.239	11.4 / 376.0	200.0	metric concordant
tcga brca	SIMPLS	10	interior tested value	Acc 0.8523 / Acc 0.8523	0.021 / 0.222	11.9 / 330.5	194.0	metric concordant
tcga hnsc methylation	kernel PLS	2	lower boundary	Acc 1.0000 / Acc 1.0000	0.011 / 0.205	10.3 / 307.2	192.0	metric concordant
tcga hnsc methylation	OPLS	2	lower boundary	Acc 1.0000 / Acc 1.0000	0.015 / 0.213	11.0 / 314.2	192.0	metric concordant
tcga hnsc methylation	PLS-SVD	1	response-rank limit	Acc 1.0000 / Acc 1.0000	0.009 / 0.235	5.2 / 386.2	198.0	metric concordant
tcga hnsc methylation	SIMPLS	2	lower boundary	Acc 1.0000 / Acc 1.0000	0.010 / 0.211	7.7 / 307.7	192.0	metric concordant
gtex v8	kernel PLS	100	upper boundary	Acc 0.9624 / Acc 0.9624	0.512 / 0.280	51.6 / 352.4	220.0	metric concordant
gtex v8	OPLS	100	upper boundary	Acc 0.9598 / Acc 0.9598	0.620 / 0.375	77.1 / 384.9	220.0	metric concordant
gtex v8	PLS-SVD	31	response-rank limit	Acc 0.9410 / Acc 0.9410	0.152 / 0.263	26.4 / 379.6	220.0	metric concordant
gtex v8	SIMPLS	100	upper boundary	Acc 0.9624 / Acc 0.9624	0.512 / 0.281	51.6 / 357.6	220.0	metric concordant
tcga pan cancer	kernel PLS	100	upper boundary	Acc 0.9358 / Acc 0.9348	0.456 / 0.277	43.8 / 365.3	216.0	metric concordant
tcga pan cancer	OPLS	100	upper boundary	Acc 0.9328 / Acc 0.9338	0.488 / 0.354	65.0 / 377.0	216.0	metric concordant
tcga pan cancer	PLS-SVD	31	response-rank limit	Acc 0.8809 / Acc 0.8809	0.123 / 0.261	23.0 / 447.4	210.0	metric concordant
tcga pan cancer	SIMPLS	100	upper boundary	Acc 0.9358 / Acc 0.9348	0.409 / 0.278	43.8 / 354.8	216.0	metric concordant
retina	kernel PLS	50	upper boundary	Acc 0.9678 / Acc 0.9678	0.098 / 0.265	58.5 / 388.2	208.0	metric concordant
retina	OPLS	20	interior tested value	Acc 0.9497 / Acc 0.9499	0.107 / 0.266	64.9 / 397.7	202.0	metric concordant
retina	PLS-SVD	11	response-rank limit	Acc 0.9273 / Acc 0.9273	0.050 / 0.272	51.3 / 428.2	206.0	metric concordant
retina	SIMPLS	50	upper boundary	Acc 0.9678 / Acc 0.9678	0.100 / 0.258	64.9 / 398.7	208.0	metric concordant
tabula	kernel PLS	50	upper boundary	Acc 0.8006 / Acc 0.8006	0.283 / 0.329	75.2 / 367.3	228.0	metric concordant
tabula	OPLS	50	upper boundary	Acc 0.7982 / Acc 0.7982	0.393 / 0.425	119.8 / 390.8	228.0	metric concordant
tabula	PLS-SVD	31	response-rank limit	Acc 0.7800 / Acc 0.7800	0.250 / 0.315	61.7 / 416.8	226.0	metric concordant
tabula	SIMPLS	50	upper boundary	Acc 0.8006 / Acc 0.8006	0.297 / 0.320	75.2 / 367.4	228.0	metric concordant
cifar100	kernel PLS	200	upper boundary	Acc 0.8883 / Acc 0.8880	14.723 / 1.012	705.2 / 676.3	532.0	metric concordant
cifar100	OPLS	200	upper boundary	Acc 0.8879 / Acc 0.8868	17.603 / 4.006	1037.2 / 1008.4	532.0	metric concordant
cifar100	PLS-SVD	99	response-rank limit	Acc 0.8740 / Acc 0.8740	6.006 / 0.675	448.4 / 685.6	532.0	metric concordant
cifar100	SIMPLS	200	upper boundary	Acc 0.8883 / Acc 0.8880	14.599 / 1.007	705.6 / 676.9	532.0	metric concordant
cbmc citeseq	kernel PLS	50	upper boundary	RMSD 1056.46 / RMSD 1056.46	0.603 / 0.292	121.6 / 372.6	256.0	metric concordant
cbmc citeseq	OPLS	50	upper boundary	RMSD 1056.88 / RMSD 1056.88	0.772 / 0.433	180.7 / 428.4	256.0	metric concordant
cbmc citeseq	PLS-SVD	10	response-rank limit	RMSD 1096.5 / RMSD 1096.5	0.177 / 0.283	61.4 / 403.1	254.0	metric concordant
cbmc citeseq	SIMPLS	50	upper boundary	RMSD 1056.46 / RMSD 1056.46	0.608 / 0.291	121.2 / 372.4	256.0	metric concordant
prism	kernel PLS	5	interior tested value	RMSD 0.54677 / RMSD 0.546448	0.134 / 0.278	93.7 / 404.8	252.0	metric concordant
prism	OPLS	2	lower boundary	RMSD 0.552116 / RMSD 0.552237	8.934 / 9.133	148.8 / 423.7	246.0	metric concordant
prism	PLS-SVD	10	interior tested value	RMSD 0.551182 / RMSD 0.551219	0.176 / 0.291	61.6 / 434.5	236.0	metric concordant
prism	SIMPLS	5	interior tested value	RMSD 0.54677 / RMSD 0.546448	0.117 / 0.291	93.7 / 392.3	252.0	metric concordant
nmr	kernel PLS	n/a	not evaluated	not evaluated in NMR protocol / not evaluated in NMR protocol	n/a	NR / NR	NR	not evaluated in NMR protocol
nmr	OPLS	n/a	not evaluated	not evaluated in NMR protocol / not evaluated in NMR protocol	n/a	NR / NR	NR	not evaluated in NMR protocol
nmr	PLS-SVD	5	interior tested value	RMSD 0.00104262 / RMSD 0.00104262	2.447 / 0.898	390.3 / 710.6	590.0	metric concordant
nmr	SIMPLS	50	interior tested value	RMSD 0.000762589 / RMSD 0.000759138	10.617 / 1.971	407.0 / 698.3	3414.0	metric concordant

S17. NMR case study

The family-selected predictive analysis and paired backend analysis answer different questions. Component selection used five paired training-only splits. The PLS-SVD candidate grid was 1, 2, 3, 5, 7, 10, 25, 50, 75, 100, 125, 150, 165, 175, 200, 250, and 300 components. The SIMPLS candidate grid was 10, 25, 50, 75, 100, 125, 150, 165, 175, 200, 250, and 300 components.

For family f, component count a, and split s = 1,...,5, let e_f,s(a) denote validation RMSD. The implementation computes m_f(a) = (1/5) sum_s e_f,s(a), chooses the smallest a_min in argmin_a m_f(a), and defines the one-standard-error threshold h_f = m_f(a_min) + sd_s[e_f,s(a_min)]/sqrt(5), where sd is the sample standard deviation across the five splits. The eligible set is E_f = {a: m_f(a) <= h_f}, and the returned count is a_selected = min(E_f).

For PLS-SVD, a_min = 5, h_f = 0.001528979123, and E_f = {5}; therefore a_selected = 5. For SIMPLS, a_min = 50, h_f = 0.0009565399754, and E_f = {50, 75, 100}; therefore a_selected = 50. Each value is selected within the evaluated grid, not asserted to be a global optimum.

The paired backend analysis changes only CPU versus CUDA within family. The deposited 165-component workflow uses the original centring-only protocol and is historical context. Predictor columns with chemical shifts strictly between 4.6 and 4.8 ppm were set to zero in both training and test predictor matrices before inner splitting or fitting. No response column was zeroed, masked, or excluded; all response metrics use all 28,355 coordinates. Main-text Figure 4 displays held-out sample AMI-00BP-8 (index 155), whose per-spectrum RMSD under 50-component SIMPLS CUDA rSVD was closest to the median across the 321 held-out spectra. This prespecified descriptive rule did not select the best-predicted or most visually concordant spectrum.

Table S12. Qualified NMR evidence. Family-selected predictive comparisons, matched solver/backend comparisons, and the deposited historical workflow answer different questions and are separated. Host memory is baseline-corrected process RSS; CUDA memory is total device use above the pre-run baseline and includes context.

Analysis	Family	Implementation	A	Predictive metric	Time s	Delta host MB	Delta GPU MB	Error detail / scope
Matched solver/backend	PLS-SVD	CPU irlba	5	RMSD 0.00104262; Q² 0.989159	264.291	3192.0	0.0	deterministic reference
Matched solver/backend	PLS-SVD	CPU rsvd	5	RMSD 0.00104262; Q² 0.989159	4.738	1091.1	0.0	vs IRLBA: rel. pred. error 6.6e-12; corr 1.000000; os=20, power=2, seed=123
Matched solver/backend	PLS-SVD	CUDA rsvd	5	RMSD 0.00104262; Q² 0.989159	0.408	1121.5	610	vs IRLBA: rel. pred. error 3.75e-11; corr 1.000000; os=20, power=2, seed=123
Matched solver/backend	SIMPLS	CPU irlba	50	RMSD 0.000762589; Q² 0.994200	350.662	3573.0	0.0	deterministic reference
Matched solver/backend	SIMPLS	CPU rsvd	50	RMSD 0.000762589; Q² 0.994200	9.795	1096.5	0.0	vs IRLBA: rel. pred. error 6.29e-11; corr 1.000000; os=20, power=2, seed=123
Matched solver/backend	SIMPLS	CUDA rsvd	50	RMSD 0.000759138; Q² 0.994253	1.508	1112.3	3576	vs IRLBA: rel. pred. error 0.00566; corr 0.999981; os=20, power=2, seed=123
Historical scientific context	PLS-SVD	deposited R/IRLBA	165	RMSD 0.000709909	447.601	3605.5	n/a	Different component count/workflow/hardware; not backend-only

S18. ImageNet exploratory stress test

The pooled archive contained 1,281,167 precomputed DINOv2 embeddings with 1,024 features and 1,000 classes. Seed 123 assigned 1,000,000 rows to training and 281,167 to a complementary holdout; this was not the canonical ImageNet split. The current-package classification experiment used label-aware float32 SIMPLS, rSVD oversampling 20, two power iterations, seed 123, and CUDA LDA. SIMPLS deflation and score

projection were host-resident, so the route is hybrid. A full-score top-5 attempt was killed at 27.1 GB process RSS; the reported run used 10,000-row prediction blocks and online metric accumulation without changing fitted scores or class decisions. The separate FAISS experiment used a different estimator and objective and remains a compression/retrieval study.

Table S13. Current exploratory ImageNet classification and representation results. Shared-path fit time is repeated only to identify the common fit and must not be interpreted as ten independent component-specific fits. The 1,000-component row is the requested boundary stress point, not a selected optimum. All classification rows are single-run feasibility estimates.

Experiment	Head / representation	A/dim.	Top-1	Top-5	End-to-end s	Host RSS MB	GPU MB	Qualification
Shared SIMPLS path	LDA	100	0.7806	0.9278	shared fit 950.2 + prediction 41.6	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	200	0.7907	0.9320	shared fit 950.2 + prediction 51.9	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	300	0.7955	0.9334	shared fit 950.2 + prediction 62.2	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	400	0.7984	0.9339	shared fit 950.2 + prediction 72.7	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	500	0.8007	0.9347	shared fit 950.2 + prediction 82.9	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	600	0.8030	0.9355	shared fit 950.2 + prediction 93.5	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	700	0.8051	0.9367	shared fit 950.2 + prediction 103.3	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	800	0.8069	0.9379	shared fit 950.2 + prediction	17236	5412	single exploratory run;

Experiment	Head / representation	A/dim.	Top-1	Top-5	End-to-end s	Host RSS MB	GPU MB	Qualification
					113.0			hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	900	0.8085	0.9387	shared fit 950.2 + prediction 124.4	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
Shared SIMPLS path	LDA	1000	0.8094	0.9393	shared fit 950.2 + prediction 134.4	17236	5412	single exploratory run; hybrid; os=20, power=2, seed=123
FAISS exact retrieval	Raw DINOv2	raw	0.6556	0.9392	510.8	19163	5894	1.00x compression; 3 timing repeats
FAISS exact retrieval	PCA-rSVD scores	200	0.6430	0.9383	337.7	12250	4906	5.12x compression; 3 timing repeats
FAISS exact retrieval	PLS-SVD using rSVD scores	200	0.6516	0.9397	216.7	9925	2502	5.12x compression; 3 timing repeats

S19. Repeated outer-partition uncertainty

MetRef, GTEx v8, and Retina used ten stratified 80/20 outer partitions with 5-fold training-only component selection. NMR used five random 80/20 outer partitions and 3-fold selection. Classification used deterministic float64 CPU IRLBA; NMR used fixed-seed float64 CUDA rSVD for feasibility. Empirical ranges are descriptive across partitions, not confidence intervals.

Table S14. Definitive repeated-partition predictive dispersion and selection stability. Values are mean (SD), empirical 2.5th-97.5th percentile range, and selected-component range.

Dataset	Family	Head	Outer n	Metric	Mean (SD)	Empirical range	Selected A range	Upper-bound freq.	Constraint
gtex v8	kernel PLS	argmax	10	accuracy	0.963437 (0.0048)	0.956072-0.968702	100-100	1.00	
gtex v8	OPLS	argmax	10	accuracy	0.961628 (0.00435)	0.955362-0.96741	100-100	1.00	
gtex v8	PLS-SVD	argmax	10	accuracy	0.942765 (0.00642)	0.929554-0.949031	31-31	1.00	rank constrained
gtex v8	SIMPLS	argmax	10	accuracy	0.963437 (0.0048)	0.956072-0.968702	100-100	1.00	
metref	kernel PLS	argmax	10	accuracy	0.748023 (0.0269)	0.706921-0.789689	22-22	1.00	
metref	kernel PLS	lda	10	accuracy	0.858757 (0.0341)	0.792938-0.900141	22-22	1.00	
metref	OPLS	argmax	10	accuracy	0.756497 (0.0245)	0.71822-0.792797	22-22	1.00	

Dataset	Family	Head	Outer n	Metric	Mean (SD)	Empirical range	Selected A range	Upper-bound freq.	Constraint
metref	OPLS	lda	10	accuracy	0.863842 (0.033)	0.813559-0.901412	22-22	1.00	
metref	PLS-SVD	argmax	10	accuracy	0.742938 (0.0222)	0.713136-0.77274	20-21	0.80	rank constrained
metref	PLS-SVD	lda	10	accuracy	0.821469 (0.0269)	0.76596-0.846186	21-21	1.00	rank constrained
metref	SIMPLS	argmax	10	accuracy	0.748023 (0.0269)	0.706921-0.789689	22-22	1.00	
metref	SIMPLS	lda	10	accuracy	0.858757 (0.0341)	0.792938-0.900141	22-22	1.00	
nmr	PLS-SVD	regression	5	RMSD	0.000798261 (7.43e-05)	0.000711782-0.0008999	50-100	0.20	
nmr	SIMPLS	regression	5	RMSD	0.000773708 (5.63e-05)	0.000701997-0.000845392	50-50	0.00	
retina	kernel PLS	argmax	10	accuracy	0.96799 (0.00151)	0.965369-0.970341	20-50	0.10	
retina	kernel PLS	lda	10	accuracy	0.968737 (0.00152)	0.965966-0.970937	20-50	0.10	
retina	OPLS	argmax	10	accuracy	0.951283 (0.00211)	0.947217-0.953131	30-40	0.00	
retina	OPLS	lda	10	accuracy	0.967143 (0.00152)	0.964131-0.96898	30-50	0.10	
retina	PLS-SVD	argmax	10	accuracy	0.926266 (0.0022)	0.923146-0.930036	11-11	1.00	rank constrained
retina	PLS-SVD	lda	10	accuracy	0.947457 (0.00204)	0.944398-0.950206	11-11	1.00	rank constrained
retina	SIMPLS	argmax	10	accuracy	0.96799 (0.00151)	0.965369-0.970341	20-50	0.10	
retina	SIMPLS	lda	10	accuracy	0.968737 (0.00152)	0.965966-0.970937	20-50	0.10	

S20. Analysis provenance

The ledger never infers a historical commit from a package version or result date. Where a run did not record its Git SHA, the source status is explicitly not recoverable. SHA-256 prefixes identify immutable result archives; full digests are retained in the machine-readable ledger. The 0.99.10 code audit is documented separately in benchmark/CODE_AUDIT_0.99.10.md and does not alter historical result provenance.

Table S15. Definitive analysis provenance. Archive paths are relative to the repository root.

ID	Authoritative output	Result archive	Version	Source status	Generating script	SHA-256 prefix
A01	Tables S11 and selected-point main figure	benchmark_results/manuscript_revision_cycle20_20260725	0.99.6	not recoverable;not inferred	benchmark/summarize_paired_backend_selected.R	8285bfe1f04b
A02	Table S10	benchmark_results/manuscript_revision_cycle15_20260725	0.99.6	not recoverable;not inferred	benchmark/benchmark_pls_package_comparison.R	a0eb50a1e208
A03	Tables S7-S8	benchmark_results/simpls_estimator_preservation_reliable_power2_final_20260725	0.99.6	not recoverable;not inferred	benchmark/benchmark_simpls_estimator_preservation.R	49417c27d832

ID	Authoritative output	Result archive	Version	Source status	Generating script	SHA-256 prefix
A04	Table S12	benchmark_results/ review_nmr_extended_selection_20260725	0.99.6	not recoverable;not inferred	benchmark/ review_nmr_extended_component_selection.R	62f5da8c637a
A05	Table S12	benchmark_results/ nmr_matched_contrasts_20260726	0.99.6	not recoverable;not inferred	benchmark/ plot_nmr_matched_contrasts.R	287f1ca7865d
A06	Table S13	benchmark_results/ imagenet_faiss_matched_1m_20260725	0.99.6	not recoverable;not inferred	benchmark/ benchmark_imagenet_faiss_matched_retrieval.R	20b28a5330fc
A07	Table S9	benchmark_results/ cv_compiled_vs_r_loop_20260725	0.99.6	not recoverable;not inferred	benchmark/ benchmark_cv_compiled_vs_r_loop.R	fcfd9eec2765
A08	Table S7a	benchmark_results/ simpls_multidataset_ablation_20260725	0.99.6	not recoverable;not inferred	benchmark/ benchmark_simpls_multidataset_ablation.R	5d1846d33ca9
A09	Table S11	benchmark_results/ float32_backend_agreement_cycle5	0.99.6	not recoverable;not inferred	scripts/ run_reviewer_precision_validation.sh	7d440d6c99f3
A10	Table S7	benchmark_results/ lda_backend_agreement_order_fixed	0.99.6	not recoverable;not inferred	scripts/ rerun_cuda_lda_package_comparison.sh	35799a8caf39
A11	Table S10	benchmark_results/ manuscript_revision_cycle43_20260726	0.99.6	not recoverable;not inferred	scripts/ run_kernel_sensitivity_after_suite.sh	655adc44784e
A12	Table S7b	benchmark_results/ simpls_vs_plssvd_shapes_20260726_cuda	0.99.6	not recoverable;not inferred	benchmark/ benchmark_simpls_vs_plssvd_shapes.R	edadcb285e08
A13	Table S8	benchmark_results/ opls_kernel_estimator_validation_verified_20260726	0.99.6	not recoverable;not inferred	benchmark/ benchmark_opls_kernel_estimator_validation.R	61ba3a1486bc
A14	Table S7	benchmark_results/ metal_validation_20260726	0.99.6	recorded by run	benchmark/metal_validation/ run_metal_validation.R	6ac2dc3565df

ID	Authoritative output	Result archive	Version	Source status	Generating script	SHA-256 prefix
A15	Table S7	benchmark_results/ opls_kernel_setting_reliability_20260726	0.99.6	not recoverable;not inferred	benchmark/ benchmark_opls_kernel_setting_reliability.R	2d61edac0d99
A16	Table S14	benchmark_results/ manuscript_revision_cycle66_20260726	0.99.6	run metadata and failures retained	benchmark/ benchmark_repeated_outer_selection.R	see archive README
A17	Figure 4/Table S12; AMI-00BP-8 #155	benchmark_results/ manuscript_revision_cycle80_20260727/nmr_qualified	0.99.6	archive and selection CSV recorded	benchmark/ benchmark_nmr_qualified_solver.R	c868770fee19
A18	Figure 5 and Table S13 classification path	benchmark_results/ manuscript_revision_cycle80_20260727/ imagenet_float32_simpls_lda_path	0.99.6	source archive recorded	benchmark/ benchmark_imagenet_float32_simpls_lda_path.R	c868770fee19

S21. Repository-only detailed material

Full component paths, per-run rows, sensitivity analyses, review-cycle figures, and the former cycle-66 expanded supplement are indexed in benchmark/MANUSCRIPT_EVIDENCE_ARCHIVE.md. Their underlying CSV, RDS, PDF, PNG, log, and session-information files remain available for audit. Compact definitive ablation results are retained in Table S7a.

Supplementary references

References are numbered as in the main manuscript. The final mapping is Retina [25], Tabula Muris [26], PRISM [27], ImageNet [28], DINOv2 [29], CIFAR-100 [30], UNI [31], and Prov-GigaPath [32].